MODELADO DE DATOS CON ESTRUCTURAS (STRUCT) Y PUNTEROS

Mateo Roberts (Documentador)

Leonel De Armas (Frontend Designer)

Gabriel Zacarias (Líder de Proyecto)

Sofia Mojica (Backend Developer)

Escuela de Educación Secundaria Técnica N°1

Laboratorio de Programación (LPR) – 5° Año 3°

Actividad 5: Modelado de Datos con Estructuras (Struct) y Punteros

Proyecto Integrador Anual: Horas que Importan 2.0

15 de octubre de 2026

Modelado de Datos con Estructuras (Struct) y Punteros

1. Introducción

El presente informe detalla la implementación de estructuras de datos (struct) y el manejo de punteros en el lenguaje C++ aplicados al proyecto "Horas que Importan 2.0". El objetivo es optimizar la gestión de la memoria RAM y organizar la información de manera heterogénea para representar entidades del mundo real, sentando las bases lógicas para el prototipo del Proyecto Integrador Anual.

2. Descripción del Modelo

Para el proyecto "Horas que Importan 2.0", se ha modelado la entidad Registro de Tiempo. Esta estructura permite almacenar la información básica de un usuario y la métrica de horas trabajadas en un solo bloque contiguo de memoria RAM. Componentes del Struct implementado:

- idUsuario (int): Identificador único del empleado o usuario.
- nombreEmpleado (char[50]): Cadena de caracteres para el nombre completo.
- horasTrabajadas (float): Métrica decimal que representa el tiempo registrado.

3. Resolución del Cuestionario Técnico

3.1 Heterogeneidad

La ventaja de agrupar los datos de un dispositivo o entidad en un struct es que permite crear un "tipo de dato personalizado". En lugar de declarar múltiples variables independientes para cada atributo de un empleado, el struct permite manejar la entidad como un solo objeto lógico. Esto facilita la organización del código, la lectura del mismo y permite la creación de arreglos de estructuras para manejar múltiples registros.

3.2 Operador Flecha

El compilador de C++ no permite la notación de punto (ptr.id) cuando la variable es un puntero porque el puntero no es el objeto en sí, sino una dirección de memoria que apunta hacia él. El operador flecha (->) es la sintaxis exclusiva que permite desreferenciar el puntero y acceder directamente al miembro de la estructura almacenado en esa dirección.

3.3 Eficiencia en RAM

El pasaje por referencia mediante punteros optimiza la memoria ya que evita la creación de una copia completa de la estructura en la pila (stack). Mientras que el pasaje por valor duplica todos los datos del objeto, el pasaje por puntero solo envía la dirección de memoria (un valor pequeño), permitiendo que la función modifique el dato original directamente en la RAM.

3.4 Buffer de Entrada

Si se omite la instrucción cin.ignore() antes de un getline, el carácter de salto de línea (\n) generado al presionar "Enter" en la lectura anterior permanece en el buffer. El cin.getline() lee ese salto de línea inmediatamente y lo interpreta como el final de la entrada, provocando que el programa se salte la lectura del nombre y deje el campo vacío.

3.5 Aplicación al Proyecto Integrador

En el proyecto "Horas que Importan 2.0", representamos la entidad Registro de Asistencia. Esta estructura es la base del componente lógico de control de tiempos, donde el idUsuario vincula la sesión con la base de datos, el nombreEmpleado identifica al sujeto y las horasTrabajadas representan la métrica de operación principal del sistema.

4. Conclusiones

La implementación de punteros y estructuras permite que el software sea más eficiente y escalable. La capacidad de manipular datos por referencia es crucial para el desarrollo de sistemas de gestión de datos donde la optimización de la RAM es prioritaria. Se ha logrado

modelar la entidad principal del proyecto, asegurando un flujo de datos correcto entre el ingreso de usuario y la memoria del sistema.

Enlace al Repositorio

https://github.com/matuteXEN6/-ACTIVIDAD-5-MODELADO-DE-DATOS-CON-ESTRUCTURAS-STRUCT-Y-PUNTEROS---LPR-5-3-A-B.git